

Programming Assignment 1

Writing a Unix Shell

Writing a Unix Shell

- A shell is a user interface for access to an operating system's services
- User (or client) issues commands to the program in the form of successive lines of text (command lines)
- The shell is just another program provided by the operating system
 - You can install additional shells, and make your own.
 - For example DOS, bash, ksh, csh, Powershell etc
- Sometimes a shell provides its own programming or scripting language
 - Has its own syntax, operators to control command execution
 - e.g., | (pipe), > (output redirector), < (input redirector), conditional ||, && statements etc.

PA1 and PA2

- **PA1**: assigned this week; **PA2**: will be assigned in week 6
- We are building our own (**not fully functional**) shell (PA1)
- Parse command line, interpret, and execute command(s) (PA1)
- We will implement some features (>, <, >>, ||, &&, |, ;)
 - logical operators, process control (PA1)
 - Interprocess communication (IPC) via pipes (PA2)
- Submission
 - Submit to Canvas a zip file
 - Folder should contain all your code, and a Makefile with an “all” target.

Evaluation

- Like all other programming assignments, graded on SoC servers (Nuros,cse-linux-01)
- Tests broken out with associated points
- Your shell should run in interactive mode
- It also needs to run with our testscripts (found in the handout directory)
- For each testscript we will run:

```
./osh -t < testscripts/testscript.txt >& tmp ; diff tmp testscripts/ea.txt ;
```

Note that the “> &” redirects stderr to stdout and is unique to C-shell. For bash you would use “2>&1”

- Some output differences due to Unix version, type of parent shell
- Will dock points for:
 - infinite loops, not capable of running interactive shell mode, or you print extraneous output.
 - Check the provided binary in “server-binary” to observe correct behavior.

Run csh shell

- To test your code, you can try to change bash shell to C-shell
- "exec csh" (or "exec tcsh", tcsh is an enhanced csh) and get a csh shell

The Handout

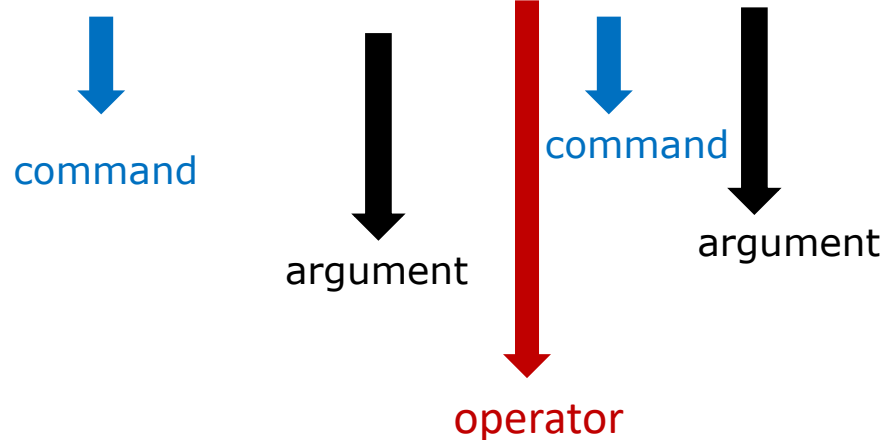
- PA1.pdf - has all the information you should read several times to understand the assignment
- The provided parser, and an associated Makefile give you a nice starting point
 - Type "Make", then run "./osh" and try it out, you'll get the debug information of a working parser
 - The README file provides some useful information about the parser. You should also look carefully at the code provided
- "server-binary" - has a precompiled binary you can run on the servers to observe the correct output of a working shell
- "testscripts" - test scripts and associated expected answer output
 - Output from your shell should match the text in ea*.txt
 - NOTE: some slight differences between running in interactive vs. test mode

Parts of input

- Side note : Anatomy of a Command

<executable name> <list of arguments> [operator] [more commands]

- `cat` testdir/list.txt | `grep` file3.txt



Approach

- Part 1 (PA1)
 - Phase 1 : Interpret input command and recognize invalid command (**done for you**)
 - Phase 2 : Execute command
 - Phase 3 : Logical operators
 - Phase 4 : Implement testing switch
- Part 2 (PA2) builds on PA1
 - Your PA1 **must work** for you to build on for PA2!
 - Phase 5 : Interprocess Communication (pipes)

Part 1: Phase 1 - input parsing

- Read, parse, and interpret command line inputs
 - (a) read the input line
 - (b) tokenize input string to command, arguments and operators
 - (c) optional store it in a format to make it easy to handle input
 - (d) identify malformed commands
- This part is done for you, you just need to use it! Check out the README in the handout folder.
- Demo: Parser

Identifying malformed commands

(i) NULL Command

eg: osh> | ls

this is wrong as we have a null command before the |

(ii) Missing Files for redirectors

eg: osh> ls >

the output redirector is expecting a file, but it is null in command

(iii) Multiple Redirectors

eg: osh> ls > file | cat

in this case, we have two output directors that can be associated to ls. This is ambiguous and thus malformed

```
[osh-parser]$ ./osh
osh> | ls
osh: Invalid NULL command
osh> ls >
osh: Expecting an output path
osh> ls > file | cat
osh: Ambiguous output redirect.
osh>
```

Phase 2 - execute the command

- Execute the command
 - Fork a new process, call exec, then wait for the child process to finish
- Steps (start simple):
 - Set aside the parser for a moment, write a simple C++ file that just forks, executes a command, and waits (see code to the right)
 - Once that's working, incorporate the parser
 - Now consider redirect operators (ignore the pipe until PA2)
 - To get the output/input to the right place you can use dup2()

```
1 #include <sys/types.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 int main()
6 {
7     pid_t pid;
8
9     /* fork a child process */
10    pid = fork();
11    if (pid < 0) { /* error occurred */
12        fprintf(stderr, "Fork Failed");
13        return 1;
14    }
15    else if (pid == 0) { /* child process */
16        printf("I am the child %d\n", pid);
17        execlp("/bin/ls", "ls", NULL);
18    }
19    else { /* parent process */
20        /* parent will wait for the child to complete */
21        printf("I am the parent %d\n", pid);
22        wait(NULL);
23        printf("Child Complete\n");
24    }
25
26    return 0;
27 }
```

Phase 3 & 4 - logical operators and switch

- In Phase 3 we implement handling of logical operators - && || ;
 - Start thinking about your commands executing in a loop. To implement logical operators
 1. Check to see if current and previous commands separated by logical operator
 2. If so, get exit status of the previous command
 3. Based on exit status, determine whether to execute current command
 4. If you need to skip the current command set the exit status of the current command to be the same as the previous command
 - This logic is sufficient to handle chains of operators
- In Phase 4 we implement a program switch for testing (-t)
 - Should not print the “osh>” prompt
 - This switch allows us to input an entire file of commands for testing like:

```
./osh -t < 1.singleCommand.txt >& tmp ; diff tmp testscripts/ea1.txt ;
```

Part 2 (PA2): Interprocess Communication

- In PA2, we'll implement interprocess communication with pipes “|” to complete the shell
- Allows us to directly pass output from one command into another rather than using a file and redirection
- e.g., `osh> ls | grep main`
- Will use a different lecture to discuss this

System Calls: `fork()` ,
`exec()` , `wait()`

fork()

- `fork()` is a system call to create a new child process
- The new process consists of a copy of the address space of the original (parent) process. So contains same instructions. Both continue from the next instruction
- After the `fork()` call, both processes (the parent and the child) continue execution with one difference:
 - In the child (new) process, `fork()` returns 0
 - In the parent process, `fork()` returns a positive number, which is the process id of the child
- If `fork()` failed, it returns -1

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     int pid;
7
8     pid = fork(); /* Spawn a new process */
9     if(pid == 0) { // child
10         printf("This is a Child generated by the first fork \n");
11     } else if(pid == -1) { // uh oh...
12         printf("fork() failed\n");
13     } else { // parent
14         printf("This is the Parent\n");
15         wait(NULL);
16     }
17 }
```

exec ()

- `exec ( )` is a system call to run a program/command.
- The `exec ( )` system call does not create a new process like
`fork ( )`
 - replaces the current process
- When the process is done, it executes an `exit ( )` which can be waited on by the parent
 - If the program succeeds, it returns 0.
 - Otherwise, it returns nonzero number.
- The parent can then create more children; or,
- if it has nothing else to do while the child runs, it can issue a `wait ( )` system call to move itself off the ready queue until the termination of the child.
- The call to `exec ( )` **does not return control** unless an error occurs.

```
1 #include <sys/types.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 int main()
6 {
7     pid_t pid;
8
9     /* fork a child process */
10    pid = fork();
11    if (pid < 0) { /* error occurred */
12        fprintf(stderr, "Fork Failed");
13        return 1;
14    }
15    else if (pid == 0) { /* child process */
16        printf("I am the child %d\n", pid);
17        execlp("/bin/ls", "ls", NULL);
18    }
19    else { /* parent process */
20        /* parent will wait for the child to complete */
21        printf("I am the parent %d\n", pid);
22        wait(NULL);
23        printf("Child Complete\n");
24    }
25
26    return 0;
27 }
```

Process Creation using fork()

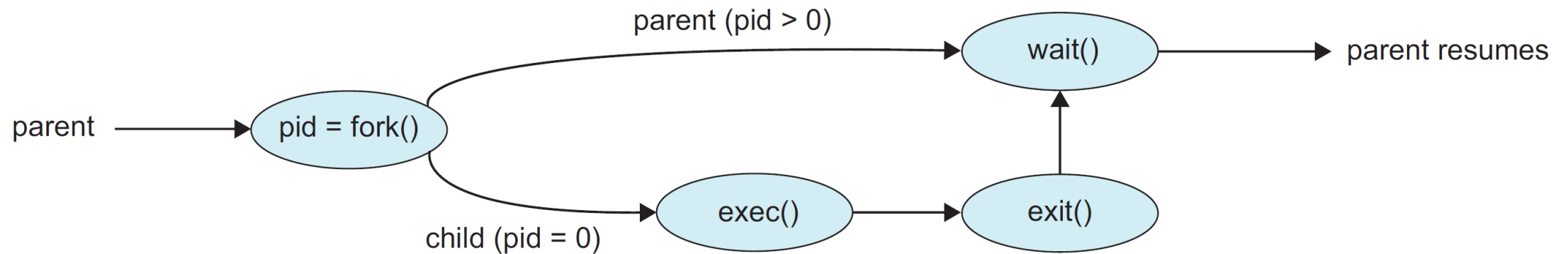


Figure 3.10 Process creation using the `fork()` system call.

dup () and dup2 ()

- `osh> ls > foo.txt`
 - Output from ls should go to foo.txt
- The `dup ( )` system call creates a copy of a file descriptor
- A **file descriptor** is a number that uniquely identifies an open **file** in a computer's operating system
- If the copy is successfully created, then the original and copy file descriptors may be used interchangeably
- They both refer to the same open file description and thus share file offset and file status flags
- Syntax: `int dup2(int oldfd, int newfd);`
 - oldfd: old file descriptor
 - newfd new file descriptor which is used by dup2() to create a copy.

Redirecting output - dup2ex.c

```
17 #include <stdlib.h>
18 #include <stdio.h>
19 #include <unistd.h>
20 #include <fcntl.h>
21
22 int main(int argc, char **argv)
23 {
24     int pid, status;
25     int fd; /* new file descriptor */
26
27     if (argc != 2) {
28         fprintf(stderr, "usage: %s output_file\n", argv[0]);
29         exit(1);
30     }
31     if ((fd = open(argv[1], O_CREAT|O_TRUNC|O_WRONLY, 0644)) < 0) {
32         perror(argv[1]); /* open failed */
33         exit(1);
34     }
35
36     printf("This goes to the standard output.\n");
37     printf("Now the standard output will go to \"%s\".\n", argv[1]);
38
39     /* this new file will become the standard output */
40     /* standard output is file descriptor 1, so we use dup2 */
41     /* to copy the new file descriptor onto file descriptor 1 */
42     /* dup2 will close the current standard output */
43
44     dup2(fd, STDOUT_FILENO);
45     close(fd);
46
47     printf("This goes to the standard output too.\n");
48     printf("And now any time I print to stdout, it goes to fd\n");
49     exit(0);
50 }
```

```
[dup]$ gcc dup2ex.c
[dup]$ ls
a.out*      dup2ex.c    dup2fork.c
[dup]$ ./a.out foo.txt
This goes to the standard output.
Now the standard output will go to "foo.txt".
[dup]$ cat foo.txt
This goes to the standard output too.
And now any time I print to stdout, it goes to fd
[dup]$
```

Output from an exed'd command - dup2fork.c

Note: in Unix systems:

0 = stdin (STDIN_FILENO)

1 = stdout (STDOUT_FILENO)

2 = stderr (STDERR_FILENO)

```
17 #include <stdlib.h>
18 #include <stdio.h>
19 #include <unistd.h>
20 #include <fcntl.h>
21
22 // function prototype
23 void runcmd(int fd, char **cmd);
24
25 int main(int argc, char **argv)
26 {
27     int pid, status;
28     int fd; // file descriptor
29     // artificial command
30     char *cmd[] = { "ls", "-al", NULL }; // don't forget the NULL
31
32     if (argc != 2) {
33         fprintf(stderr, "usage: %s output_file\n", argv[0]);
34         exit(1);
35     }
36
37     // open the file given on the command line
38     if ((fd = open(argv[1], O_CREAT|O_TRUNC|O_WRONLY, 0644)) < 0) {
39         perror(argv[1]); /* open failed */
40         exit(1);
41     }
42     // printf still actually goes to stdout
43     printf("writing output of the command %s to \"%s\"\n", cmd[0], argv[1]);
44
45     runcmd(fd, cmd); /* run the command, sending the std output to fd */
46
47     printf("all done!\n");
48     exit(0);
49 }
```

```
57 void runcmd(int fd, char **cmd)
58 {
59     int status;
60
61     switch (fork()) {
62     case 0: /* child */
63         dup2(fd, STDOUT_FILENO); // map stdout to fd
64         execvp(cmd[0], cmd);
65         perror(cmd[0]); /* bad execvp */
66         exit(1);
67
68     default: /* parent */
69         while (wait(&status) != -1) ; // reaping my zombies
70         break;
71
72     case -1: // bad fork, bad!
73         perror("fork");
74     }
75     return;
76 }
```

Final notes

- Don't wait, get started now
- You must have a working PA1 to do PA2
 - I won't provide you with a working PA1
- Be sure you have an easy-to-use mechanism for testing on the SoC servers
- Start with small tests, then add capability for more complicated commands
- When debugging, remember computers do no more, and no less, than exactly what they're told to do.
 - So if they're doing the wrong thing, your code is telling it to do the wrong thing.

Suggestion

- You are encouraged to come with your own logic and solution
- If you have no idea, below is a suggested structure

F1: Define a function to handle strings for `execvp` or `execlp`: String to char

F2: Define a function to execute the input commands

Main:

- If `argv[1] == -t`
 - do not print "osh>";
 - call parser
 - execute the commands: call F2
- Else:
 - print "osh>";
 - call parser
 - get command input and execute: call F2

F2

for each command:

```
pid_t pid;
```

```
pid = fork();
```

```
if (pid == 0){// child process
```

```
    Load in the command (ls, cat, echo,...), cin-file, cout-file;
```

```
    Use dup2 function to set the input and output correctly;
```

```
    If (this command needs to be executed){
```

```
        Call F1 function (previous slide), prepare exec function inputs;
```

```
        Call execvp/execlp function;
```

```
    }
```

```
else{// parent process
```

```
    wait(&status);
```

```
    Check next_command_mode {if on_success, check status and decide whether  
    next command needs to be executed;
```

```
    If always ...
```

```
    If on_fail ...}
```

```
}
```

PA1

- `exec()` status
 - Return values from `wait(&status)` and `WEXITSTATUS(status)`
- File:
 - Output file: `open(cout_file, O_WRONLY | O_TRUNC | O_CREAT, 0644)`
 - Input file: `open(cin_file, O_RDONLY)`
 - Append: `open(cout_file, O_WRONLY|O_APPEND| O_CREAT, 0644)`
- `O_CREAT`. To create the file if it doesn't exist already.
- `O_TRUNC`. If the file does exist, to "truncate" it to zero bytes, erasing what was there.
- `O_WRONLY`. Open for writing only.
- `O_RDWR`. Open for reading and writing.
- A permission of 644 means
 - (4+2) = read/write permission for the owner
 - (4) = read permission for the group
 - (4) = read permission for all other users